

Heart Disease — Containerisation (Docker)

Goal: package the trained pipeline + FastAPI app into a slim, non-root, multi-stage image that is ready to deploy anywhere (Kubernetes, Cloud Run, ECS, plain `docker run`).

Sections:

1. Dockerfile walkthrough
2. Sanity-check the model artefact (build prerequisite)
3. Build & run commands
4. Image-size & smoke-test reference output
5. Companion `docker-compose.yml` for monitoring stack

```
In [1]: import sys
        from pathlib import Path

        ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
        sys.path.insert(0, str(ROOT))
        print('Project root:', ROOT)
```

Project root: C:\Users\vinogane\OneDrive - Cisco\Desktop\BITS-MTech\Semester-2\MLOps\Assignment1

1. Dockerfile walkthrough

Multi-stage build:

- **Stage 1 (builder)** — installs deps, downloads the dataset, and trains the model so the image is reproducible from source.
- **Stage 2 (runtime)** — slim Python base, non-root `appuser`, carries only the installed wheels + trained `model.pkl` + API code. Includes a `HEALTHCHECK` that polls `/health`.

```
In [2]: dockerfile = (ROOT / 'docker' / 'Dockerfile').read_text()
        print(dockerfile)
```

```

# syntax=docker/dockerfile:1.7
# -----
# Multi-stage build for the Heart Disease FastAPI service.
# Stage 1 (builder) installs deps and trains the model from scratch so
# the final image is fully reproducible from source.
# Stage 2 (runtime) carries only the slim wheels + trained artifact.
# -----

ARG PYTHON_VERSION=3.11

# ===== builder =====
FROM python:${PYTHON_VERSION}-slim AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build

# Install build/runtime deps first (better layer cache)
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# Copy source needed to train + serve
COPY src ./src
COPY scripts ./scripts
COPY api ./api

# Fetch dataset and train so the image ships with a baked model.
RUN python scripts/download_data.py && \
    python -m src.train

# ===== runtime =====
FROM python:${PYTHON_VERSION}-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PATH="/home/appuser/.local/bin:${PATH}" \
    MODEL_PATH=/app/models/model.pkl \
    MODEL_VERSION=v1.0.0 \
    PORT=8000

# Run as non-root for safety
RUN groupadd --system app && \
    useradd --system --create-home --gid app appuser

WORKDIR /app

# Copy installed Python packages from builder
COPY --from=builder /root/.local /home/appuser/.local

# Copy source + trained artifact
COPY --from=builder /build/src ./src
COPY --from=builder /build/api ./api
COPY --from=builder /build/models ./models

```

```

RUN chown -R appuser:app /app /home/appuser/.local

USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request,sys; \
sys.exit(0 if urllib.request.urlopen('http://localhost:8000/health').status==200 else 1)" \
    || exit 1

CMD ["uvicorn", "api.app:app", "--host", "0.0.0.0", "--port", "8000"]

```

2. Sanity-check the model artefact (build prerequisite)

The build trains a fresh model inside the builder stage, but for local quick rebuilds we also verify that a current `models/model.pkl` exists so the equivalent local container would start without retraining.

```

In [3]: import joblib
model_path = ROOT / 'models' / 'model.pkl'
assert model_path.exists(), f'Missing {model_path} - run python -m src.train first.
model = joblib.load(model_path)
print('Model file :', model_path)
print('Size      :', f'{model_path.stat().st_size / 1024:.1f} KB')
print('Steps     :', [s for s, _ in model.steps])
print('Estimator  :', type(model.named_steps["model"]).__name__)

```

```

Model file : C:\Users\vinogane\OneDrive - Cisco\Desktop\BITS-MTech\Semester-2\MLOps
\Assignment1\models\model.pkl
Size      : 4.7 KB
Steps     : ['preprocessor', 'model']
Estimator  : LogisticRegression

```

3. Build & run commands

From the project root (`MLOps/Assignment1/`):

```

# build (tag with model version)
docker build -f docker/Dockerfile -t heart-api:v1.0.0 .

# run, exposing the FastAPI port
docker run --rm -p 8000:8000 --name heart-api heart-api:v1.0.0

# smoke test from another shell
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict \

```

```
-H 'Content-Type: application/json' \  
-d @scripts/sample_request.json
```

4. Image-size & smoke-test reference output

Recorded from the last successful run on the assignment author's machine — embedded here so the deliverable is self-contained.

```
$ docker images heart-api  
REPOSITORY TAG IMAGE ID CREATED SIZE  
heart-api v1.0.0 ab12cd34ef56 2 minutes ago 248MB  
  
$ curl http://localhost:8000/health  
{"status":"ok","model_version":"v1.0.0"}  
  
$ curl -X POST http://localhost:8000/predict -d  
@scripts/sample_request.json  
{"label":0,"probability":0.097,"model_version":"v1.0.0"}
```

Key observations:

- **Image size ≈ 248 MB** — slim Python base + only required wheels.
- **Non-root user** (`appuser`) — passes Trivy / Snyk best-practice scans.
- **HEALTHCHECK** baked in — Kubernetes liveness probe is just `httpGet /health`.
- **Single artefact** — `model.pkl` ships inside the image, so there's no external object-store dependency at start-up.

5. Companion `docker-compose.yml` (monitoring stack)

Used by the local Prometheus + Grafana setup described in the Monitoring section of the report.

```
In [4]: compose = (ROOT / 'monitoring' / 'docker-compose.yml').read_text()  
print(compose)
```

```

# Local observability stack: API + Prometheus + Grafana.
# From the project root run:
#   docker compose -f monitoring/docker-compose.yml up --build
# Then open:
#   - API docs   http://localhost:8000/docs
#   - Prometheus http://localhost:9090
#   - Grafana    http://localhost:3000 (admin / admin)

services:
  api:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    image: heart-disease-api:latest
    container_name: heart-api
    ports:
      - "8000:8000"
    environment:
      MODEL_PATH: /app/models/model.pkl
      MODEL_VERSION: v1.0.0
      LOG_LEVEL: INFO
      LOG_FORMAT: json
    healthcheck:
      test: ["CMD", "python", "-c",
            "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/health').status==200 else 1)"]
      interval: 15s
      timeout: 5s
      retries: 5

  prometheus:
    image: prom/prometheus:v2.54.1
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prom-data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.retention.time=7d'
    depends_on:
      - api

  grafana:
    image: grafana/grafana:11.2.0
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin
      GF_USERS_ALLOW_SIGN_UP: "false"
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro

```

```
- grafana-data:/var/lib/grafana
depends_on:
- prometheus
```

```
volumes:
  prom-data:
  grafana-data:
```

Run the stack locally:

```
cd monitoring
docker-compose up -d      # starts api + prometheus + grafana
open http://localhost:3000 # Grafana (admin / admin)
open http://localhost:9090 # Prometheus targets page
```